Random Orthogonal Additive Filters: A Solution to the Vanishing/Exploding Gradient of Deep Neural Networks

Andrea Ceni

***Abstract*— Since the recognition in the early 1990s of the vanishing/exploding (V/E) gradient issue plaguing the training of neural networks (NNs), significant efforts have been exerted to overcome this obstacle. However, a clear solution to the V/E issue remained elusive so far. The pursuit of approximate dynamical isometry, i.e., parameter configurations where the singular values of the input–output Jacobian (IOJ) are tightly distributed around 1, leads to the derivation of an NN's architecture that shares common traits with the popular residual network (ResNet) model. Instead of skipping connections between layers, the idea is to filter the previous activations orthogonally and add them to the nonlinear activations of the next layer, realizing a convex combination between them. Remarkably, the impossibility of the gradient updates to either vanish or explode is demonstrated with analytical bounds that hold even in the infinite depth case. The effectiveness of this method is empirically proved by means of training via backpropagation an extremely deep multilayer perceptron (MLP) of 50k layers, and an Elman NN to learn long-term dependencies in the input of 10k time steps in the past. Compared with other architectures specifically devised to deal with the V/E problem, e.g., LSTMs, the proposed model is way simpler yet more effective. Surprisingly, a single-layer vanilla recurrent NN (RNN) can be enhanced to reach state-of-the-art performance, while converging super fast; for instance, on the psMNIST task, it is possible to get test accuracy of over 94% in the first epoch, and over 98% after just ten epochs.**

***Index Terms*— Deep learning, exploding gradient, machine learning, recurrent neural networks (RNNs), vanishing gradient.**

I. Introduction

ARTIFICIAL deep neural networks (NNs) are computational models that have gathered massive interest in the last decade [1]. Usually, NNs are composed of many simple units (neurons) organized in multiple layers that nonlinearly interact with each other via trainable parameters, also called weights. Roughly speaking, training an NN to solve a task means adapting its parameters to fit an unknown function mapping between raw input and the output labels forming the data points from which to generalize.

Although several algorithms exist for training NN models, the backpropagation algorithm [2], [3] with stochastic gradient descent (SGD) has been established as a standard for supervised learning. The backpropagation is a clever procedure to assess how sensitive the output error is with respect to a given parameter of the model; thus, each parameter is modified proportionally to this measure of sensitivity. The computation involves the sequential product of many matrices whose resulting norm, similar to the product of many real numbers, can shrink to vanish or expand to infinity exponentially with depth. This problem has been known in the literature as the vanishing/exploding (V/E) gradient issue. The V/E issue was first recognized in the context of recurrent NNs (RNNs) in the 1990s [4], and then, after the advent of increasingly deep architectures, it became evident also in feedforward NNs (FNNs) [5].

A. Related Works

Extensive works have been made to combat the V/E gradient issue. Some of the main strategies are reported in this section. Based on the particular choice of activation function, it has been proved that some weight initialization strategies, known as Xavier-init [5] and Kaiming-init [6], generate favorable statistics in the neuronal activations that alleviate the issue. The V/E dynamics have been recognized to be stronger in "squashing" activation functions, e.g., tanh, while being gentler on hard-nonlinear activation functions, e.g., ReLUs. Moreover, true zero neuronal activations induced by ReLUs promote sparsity into the network, which, in turn, has been recognized to be beneficial for backpropagating errors, as long as some neural paths remain active [7]. Nevertheless, the issue, although mitigated, is still present with ReLUs. Moreover, a mean-field theory analysis on linear FNNs revealed the effectiveness of orthogonal weights initialization schemes [8], and provided that an NN operates close to the so-called *edge of chaos* [9], on nonlinear FNNs as well. In fact, the V/E gradient issue can be solved by promoting the NN to operate in an approximate *dynamical isometry* regime, i.e., a configuration where the distribution of the singular values of the NN's input–output Jacobian (IOJ) remains circumscribed around 1 without spreading too far from it [10]. This condition would ensure that signals can flow back and forth along the layered architecture. However, containing the learning process into an approximate dynamical isometry regime appears to be hard in general FNNs, due to the nonlinearities between layers. Orthogonal initialization schemes [11], although not directly solving the V/E problem, have been successfully implemented to train extremely deep NNs. Additionally, gradient clipping procedures [12] have been extensively used to maintain the

Received 12 September 2022; revised 1 September 2023, 16 September 2024, 17 November 2024, and 8 December 2024; accepted 2 February 2025. Date of publication 26 February 2025; date of current version 4 June 2025. This work was supported in part by the Mathematical Sciences EPSRC (University of Exeter, U.K.), in part by the Tuscany Regional Government under the "Progetti speciali COVID-19" program, project "MyBreathingHeart", under Grant CUP I55F20000610007, in part by EU-EIC EMERGE under Grant 101070918, in part by NEURONE, a project funded by the European Union-Next Generation EU, under Grant M4C1 CUP I53D23003600006, and in part by PRIN 2022 under Grant 20229JRTZA.

The author is with the Department of Computer Science, University of Pisa, 56127 Pisa, Italy (e-mail: andrea.ceni@di.unipi.it).

Digital Object Identifier 10.1109/TNNLS.2025.3538924

2162-237X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

gradients at a controlled magnitude, while not addressing the vanishing problem. Similarly, activating the gradients via parameterized tanh-like functions has been demonstrated to curb the V/E issue [13]. Besides, normalizing the incoming input from a layer to the consecutive one, in order to avoid the saturating tails, has been shown to accelerate training while improving the performance [14]. However, these are passive strategies that do not aim to prevent the V/E gradient issue to occur. Another breakthrough comes from Highway Network [15], an FNN architecture, inspired by the popular LSTM RNN model, provided with trainable gating mechanisms to regulate information flow. Finally, residual networks (ResNets) [16], a gateless version of Highway Networks, have revolutionized the field of computer vision. These architectures provide shortcut paths, directly connecting nonconsecutive layers. Despite the empirical achievements of ResNets, theoretical results establishing the effective solution of the V/E gradient problem for those architectures are still missing, and ResNets still heavily rely on batch normalization for very deep architectures [17]. In practice, it is not rare in the literature to encounter several of the aforementioned techniques alchemically combined together to train deep learning models.

RNNs have been especially known for the difficulty of training them to learn long-term dependencies. Probably, the most popular solution proposed to combat the V/E issue of RNNs is represented by gated RNN architectures, e.g., as LSTMs [18], and GRUs [19]. However, despite the increased computational complexity of these models, they still suffer from instabilities when trained on very long sequences. Similar to FNNs, orthogonal initialization of the recurrent weights has been investigated and proved to be effective for RNNs [20]. Arjovsky et al. [21] go beyond the mere orthogonal initialization and propose to parametrize the recurrent weights in the space of unitary matrices, thereby ensuring the orthogonality condition during the learning process. Then, Wisdom et al. [22] further improve such a model allowing the learning dynamics to access the whole submanifold of the unitary matrices. However, these methods come at the expense of considerable computational effort. After these achievements, a burst of works has followed in the footsteps of constraining the recurrent weights to be orthogonal or approximately such [23], [24] (and references therein) trying to restrain the run time. Although it is not understood how, and to what extent, these restrictions imposed on recurrent weights impact the learning dynamics, it is widely recognized in the literature that imposing unitary constraints on the recurrent matrix affects its expressive power [25], [26]. Alternatively to orthogonal constraints, IndRNN [27] is a recently proposed RNN model where hidden-to-hidden weights are constrained to diagonal matrices nonlinearly stacked in two or more layers as a single recurrent block. V/E gradient is, thus, prevented via bounding the coefficients of those diagonal matrices in a suitable range, which depends on the length of the temporal dependencies the IndRNN is aimed to learn. Another option is to prevent the V/E issue at a fundamental level by constraining the recurrent matrix to be constant, i.e., untrained. An emblematic example is given by reservoir computing machines [28]. Typically, the recurrent weights are randomly generated and left untouched, tuning them just at a hyperparameter level, e.g., rescaling the spectral radius, and only an output readout is trained. Other important developments come from considering continuous-time RNNs as ODEs. For instance, AntisymmetricRNN [29] uses skew-symmetric recurrent matrices to ensure stable dynamics. Based on control theory results that ensure global exponential stability, LipschitzRNN [30] extends the pool of recurrent matrices where the learning occurs outside the sole space of skew-symmetric matrices. Another promising architecture is coRNN [31], derived from a second-order ODE modeling a network of oscillators. Other interesting works include models provided with memory cells, e.g., LMUs [32] and NRUs [33], or with external memories [34], skip connection-based RNNs [35], [36], [37], and many recent others [38], [39], [40], [41], [42], [43].

Despite these advancements, many solutions involve auxiliary techniques or increased model complexity, leaving room for approaches that inherently mitigate the V/E gradient problem without such dependencies. This gap motivates the development of methods that ensure stable gradient dynamics by design, enabling the training of extremely deep or recurrent architectures efficiently and effectively.

B. Original Contributions

In this article, a solution to the V/E gradient issue of deep learning models trained via SGD methods is proposed. Empirically, it is demonstrated how to: 1) enhance the vanilla FNN model to be trainable for extremely deep architectures, e.g., 50k layers (see Section II-D), without relying on auxiliary techniques as gradient clipping, batch normalization, nor skipping connections and 2) enhance the vanilla RNN model to learn very long-term dependencies, outperforming the great majority of RNN models (among which LSTMs) in benchmark tasks, such as the copying memory, the adding problem, and the permuted sequential MNIST (see Section IV).

Based on the same intuition, two NN models are proposed: 1) roaFNN, which stands for *random orthogonal additive FNN* and 2) roaRNN, which stands for *random orthogonal additive RNN*. Those models result in slight variations of, respectively, a multilayer perceptron (MLP), and an Elman network. The simplicity of these models permits a formal mathematical analysis of their gradient update dynamics, which even holds for the case of infinite depth. In particular, both lower and upper bounds are provided for: 1) the maximum singular value of the IOJ of the roaFNN model (Theorem 1), demonstrating the impossibility for it to either explode or converge to zero and 2) the entire set of singular values of the IOJ of the roaRNN model (Theorem 2), proving that a roaRNN evolves in an approximate dynamical isometry regime by design.

Remarkably, these theoretical results are achieved without any constraint on the parameters of the model.

II. Dynamical Isometry in Nonlinear Deep FNNs

In this section, we provide the reader with the necessary background to understand the backpropagation algorithm and the V/E gradient problem. Then, the proposed solution is introduced and tested on a proof of concept task.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

A. Background

An MLP is described via the following equations:

$$y_l = W_l x_l + b_l \tag{1}$$

$$x_{l+1} = \phi(y_l), \quad l = 0, 1, \ldots, L-1. \tag{2}$$

Throughout this article, vectors are assumed column vectors unless otherwise specified. That said, $x_l \in \mathbb{R}^{N_l}$ is the vector containing the neuronal activations of the lth layer, and $x_L \in \mathbb{R}^{N_L}$ is the output vector corresponding to the input vector $x_0 \in \mathbb{R}^{N_0}$. Matrices $W_l \in \mathbb{R}^{N_{l+1} \times N_l}$ represent the connection weights from neurons of the lth layer to the $l+1$th layer, and $b_l \in \mathbb{R}^{N_{l+1}}$ are the biases. The function ϕ is called *activation function*, and it is applied componentwise to each component of the vector y_l. This function must be nonlinear and usually is required to be monotonically nondecreasing, continuously differentiable (almost everywhere), and often, bounded. Common choices of ϕ are hyperbolic tangent and ReLU. Given a training dataset $\mathcal{D} = \{(X(i), Y(i))\}$ of input–output sample pairs, training of (1) and (2) is achieved by tuning its parameters, i.e., all the matrices of weights W_l and vector of biases b_l, such that the "distance" between the computed output $x_L(i)$ corresponding to the input $X(i)$ and the desired output $Y(i)$ is as low as possible on the overall training set. This translates into minimizing the sum of the distances on each input–output sample, $\mathcal{L}(\mathcal{D}) = (1/\#\text{samples}) \sum_i \mathcal{L}(Y(i), x_L(i))$; such distance is commonly called the *loss function*; in this article, it will be denoted as $\mathcal{L}$. Common choices are given by the mean squared error (mse) when dealing with regression tasks or the cross-entropy with classification tasks. The former, for a given input–output sample, reads $\mathcal{L}(Y(i), x_L(i)) = \|Y(i) - x_L(i)\|^2$, where $\|a\|$ denotes the Euclidean norm of the vector a. Note that the computed output $x_L(i)$ is a function of all the parameters of the model and the input $X(i)$, with a nested dependence on the parameters, which increases as the depth of the model. Therefore, considering that the data points $[X(i), Y(i)]$ are fixed, we can actively optimize the loss as a function of the parameters of the model, $\mathcal{L} = \mathcal{L}(W_0, b_0, \ldots, W_{L-1}, b_{L-1})$. At a first-order approximation, each parameter contributes to the loss in proportion to the derivative of the loss function with respect to it. Thus, to minimize the loss, the core idea is to modify each parameter moving toward the opposite direction pointed by the gradients. The derivative of the mse loss with respect to the output is the following vector $E(i) = -2(Y(i) - x_L(i))$. This will be called the *error vector* of the specific input–output sample $[X(i), Y(i)]$. However, in the following, it will be often denoted shortly as E, i.e., dropping the relation with the specific input–output sample. The derivative of the loss function with respect to the weights connecting the sth layer to the $(s+1)$th layer can be compactly written as the following outer product[1]:

$$\frac{\partial \mathcal{L}}{\partial W_s}(i) = \left[\text{diag}(d_s)\mathcal{B}_s(E(i))\right] \otimes x_s, \quad s = L-1, \ldots, 0 \tag{3}$$

[1]Outer product of vectors $a \in \mathbb{R}^N$ and $b \in \mathbb{R}^M$ is an $N \times M$ matrix of 1-rank defined as $(a \otimes b)_{ij} = a_i b_j$. Equivalently, considering a, b, respectively, matrices of dimensions $(N, 1), (M, 1)$, and then using the standard matrix multiplication operation, $a \otimes b = ab^{\mathrm{T}}$.

where $\text{diag}(d_s)$ is the diagonal matrix whose diagonal is the following vector:

$$d_s = \phi'(y_s) \tag{4}$$

and the linear operator $\mathcal{B}_s$ backpropagates the error vector from the output layer up to the $(s+1)$th layer, as follows:

$$\mathcal{B}_s(E(i)) = \left(\frac{\partial x_L}{\partial x_{s+1}}\right)^{\mathrm{T}} E(i) \tag{5}$$

where the subscript T denotes the transpose. The matrix $(\partial x_L / \partial x_{s+1})$ can be expressed as the product of $L - s - 1$ matrices

$$\frac{\partial x_L}{\partial x_{s+1}} = \frac{\partial x_L}{\partial x_{L-1}} \frac{\partial x_{L-1}}{\partial x_{L-2}} \cdots \frac{\partial x_{s+2}}{\partial x_{s+1}} \tag{6}$$

$$\frac{\partial x_{l+1}}{\partial x_l} = \text{diag}(d_l) W_l. \tag{7}$$

Derivatives of the loss function with respect to biases b_s can be obtained simply by omitting the outer product with the vector x_s in (3). In gradient-based learning methods, the weights W_s are updated by subtracting the matrix (3) rescaled by the learning rate μ, that is,

$$W_s(i) = W_s(i-1) + \Delta W_s(i) \tag{8}$$

$$\Delta W_s(i) = -\mu \frac{\partial \mathcal{L}}{\partial W_s}(i). \tag{9}$$

To accelerate the learning process, often training is accomplished by dividing the dataset into batches of m training samples $\mathcal{D}_j = \{(X(j), Y(j)), \ldots, (X(j+m-1), Y(j+m-1))\}$. In that case, the correction matrix (9) is averaged over a batch as follows:

$$\Delta W_s(j) = -\frac{\mu}{m} \sum_{i=0}^{m-1} \frac{\partial \mathcal{L}}{\partial W_s}(j+i). \tag{10}$$

Whether through mini-batches (10) or one-by-one samples (9), the update rule (8) realizes an SGD algorithm leading to a journey in the parameter space that will converge to a (hopefully low enough) minimum point of the loss function.

B. V/E Gradient Problem

Definition 1: Let us be given an input $x_0 = X(i)$, then it is defined through (1) and (2) the sequence of preactivations $y_1, \ldots, y_{L-1}$ corresponding to $X(i)$, and thus, the sequence of derivatives $d_1, \ldots, d_{L-1}$ via (4). It is well-defined via (7) the product matrix

$$\frac{\partial x_L}{\partial x_1}(i) = \frac{\partial x_L}{\partial x_{L-1}} \frac{\partial x_{L-1}}{\partial x_{L-2}} \cdots \frac{\partial x_2}{\partial x_1} = \prod_{l=L-1}^{1} \text{diag}(d_l) W_l \tag{11}$$

which is called the IOJ of the NN evaluated on the input $X(i)$.

Remark 1: The chain rule of calculus allows us to compute the derivative of a loss with respect to an earlier state by taking the product of derivatives at each time step. These derivatives are represented by matrices that reflect the sensitivity of the current state to the previous one. Thus, the total sensitivity of the loss can potentially vanish or explode easily as the temporal depth increases. In fact, note that the linear operator mapping the error $E(i)$ backwardly to the first layer is nothing

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

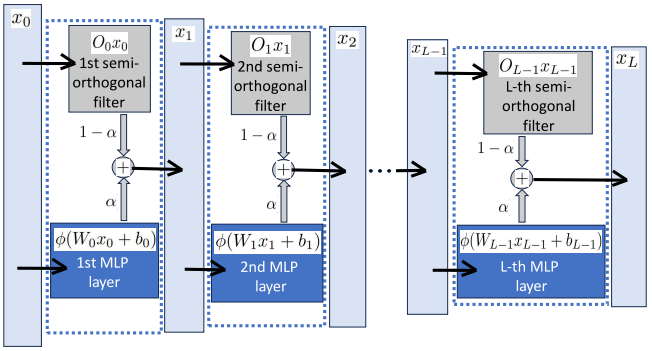


Fig. 1. Scheme of roaFNN, defined in (12). Gray parts are untrained, and blue parts are adapted with SGD via backpropagation. Light blue boxes represent the vectors of neuronal activations at each layer, from input x_0 to output x_L.

but the transpose of the IOJ since (5) for $s = 0$ reads $\mathcal{B}_0(E(i)) = ((\partial x_L/\partial x_1)(i))^{\mathrm{T}}E(i) = (\prod_{l=1}^{L-1} W_l^{\mathrm{T}}\,\mathrm{diag}(d_l))E(i)$. The IOJ is a product of $L-1$ matrices. Similar to the product of $L-1$ real numbers, as the depth L increases, the IOJ typically tends to have a norm either progressively closer to zero or exploding to infinity, regardless of the input $X(i)$. This, respectively, vanishes the update rule (8) making the learning dynamics effectively immobile for those weights far from the output, or making it explode provoking numerical overflows. In both cases, training very deep NNs becomes unfeasible.

C. Proposed Solution

The key idea is to realize between all layers a convex combination between the standard MLP transformation and a semiorthogonal transformation (see Fig. 1). In formula, (1) and (2) are modified by adding random semiorthogonal matrices O_l as follows:

$$x_{l+1} = \alpha\phi(W_l x_l + b_l) + (1-\alpha)O_l x_l, \quad l = 0, 1, \ldots, L-1 \tag{12}$$

where α is a parameter to set accordingly to the depth of the model, and O_l are randomly generated semiorthogonal matrices with dimensions matching those of W_l. Model (12) is called a *random orthogonal additive FNN (roaFNN)*.

Definition 2: A matrix $A \in \mathbb{R}^{N\times M}$ is called *semiorthogonal* if either $A^{\mathrm{T}}A = I_M$ (when $M < N$), or $AA^{\mathrm{T}} = I_N$ (when $M \geq N$), where I_n is the identity matrix of dimension n.

Remark 2: The condition $A^{\mathrm{T}}A = I_M$ implies that $\|Ax\| = \|x\|$ for all $x \in \mathbb{R}^M$. In fact, $\|Ax\|^2 = (Ax)^{\mathrm{T}}(Ax) = x^{\mathrm{T}}A^{\mathrm{T}}Ax = x^{\mathrm{T}}x = \|x\|^2$. Analogously, the condition $AA^{\mathrm{T}} = I_N$ implies that $\|A^T x\| = \|x\|$ for all $x \in \mathbb{R}^N$. In either cases, it holds that $\|A\| = 1$, denoting $\|A\|$ the matrix norm of A induced by the Euclidean norm of vectors, i.e., the maximum singular value.

1) How to Generate Random Semiorthogonal Matrices: There are a few ways to generate random semiorthogonal matrices. The method used in this article is to first generate a random matrix D of the desired dimension $N \times M$ with i.i.d. uniformly random entries in $(-1, 1)$, then perform a QR decomposition of D = QR, and take the resulting $N \times M$ semiorthogonal matrix Q. However, note that this method does not ensure the uniform exploration of the space of orthogonal matrices [44].

Since the semiorthogonal matrices of (12) are kept constant, the derivative of the mapping from layer l to layer $l+1$ with respect to weights is simply rescaled by α. Consequently, (3) is rescaled as follows[2]:

$$\frac{\partial \mathcal{L}}{\partial W_s}(i) = \alpha\big[\mathrm{diag}(d_s)\mathcal{B}_s(E(i))\big] \otimes x_s, \quad s = L-1, \ldots, 0 \tag{13}$$

where $\mathcal{B}_s(E(i))$ is defined as in (5), but with the major innovation that the structure of the Jacobian in (7) now reads

$$\frac{\partial x_{l+1}}{\partial x_l} = \alpha\,\mathrm{diag}(d_l)W_l + (1-\alpha)O_l. \tag{14}$$

Roughly speaking, by a continuity argument when α goes to zero, the Jacobian (14) tends to the semiorthogonal matrix O_l, and consequently, it inherits the isometric property in the limit of $\alpha \to 0$. The Jacobian between two consecutive layers is used to transfer the error backwardly through the two layers. Hence, the form of the Jacobian as a convex combination of an orthogonal matrix and the vanilla FNN's Jacobian reveals the architectural bias enforced by the proposed model, i.e., an approximately isometric transfer of the error flow, controllable via the hyperparameter α. *The insight is that with an appropriate choice of α (reciprocally depending on the depth L), we can control the spread of the singular value distribution of the IOJ, thereby promoting approximate dynamical isometry.* The challenge is, thus, to find the sweet spot for α depending on L. As stated in Theorem 1, whose proof can be found in Appendix A, the choice of $\alpha = (L-1)^{-1}$, or at least in a neighborhood of $(L-1)^{-1}$, prevents the IOJ to explode or vanish.

Theorem 1: Let us consider (12) with an activation function ϕ such that its derivative assumes values in $[0, r]$. Let us denote $\sigma_{\max} := \max_l \|W_l\|$, i.e., the maximum weights norm among all layers. If we parametrize $\alpha = \rho(L-1)^{-1}$, with a positive real-valued ρ, then the maximum singular value of the IOJ of (12) admits the following L-independent upper bound:

$$\left\|\frac{\partial x_L}{\partial x_1}\right\| \leq \exp(\rho(r\sigma_{\max} - 1)). \tag{15}$$

Moreover, if $\rho < ((L-1)/(1+r\sigma_{\max}))$, and assuming the sequence of layers' dimensions (i.e., the number of neurons in each layer) from input to output is nonincreasing, then it holds the following L-independent lower bound:

$$\exp(-\rho(1+r\sigma_{\max})) \leq \left\|\frac{\partial x_L}{\partial x_1}\right\|. \tag{16}$$

Remark 3: Note that, via the parametrization of $\alpha = \rho(L-1)^{-1}$, the bounds of Theorem 1 hold even in the case of infinite depth $L \to \infty$. This implies that even with a prohibitively large number of layers, all the parameters of a roaFNN model are updated in a controlled way depending on the three hyperparameters: $\sigma_{\max}, r$, and ρ. Moreover, the lower

[2]This rescaling has the consequence of effectively reducing the size of the gradients by a factor of α. That is the reason why learning rates in the following simulations are higher than usually set in the literature.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

bound (16) ensures that when the depth approaches infinity, the product of Jacobians will not converge to the null matrix. Usually, the expressive power of an NN model increases with depth. However, when the depth increases, also the risk of instabilities in training the model increases. With the proposed model, we recognized a way to allow the training of extremely deep architectures overcoming the usual instabilities that arise with standard vanilla FNN models. This is possible due to the action of the random orthogonal filters that preserve the stability of the information flow both in the forward and the backward passes.

Remark 4: Theorem 1 ensures that a standard choice of $\rho = 1$ suffices to prevent the IOJ from either vanish or explode, but the learning dynamics might be heavily influenced by the particular choice of ρ. Therefore, for optimal performance, the parameter ρ should be optimized for the task at hand.

Remark 5: The particular case of the semi-identity matrix in place of random semiorthogonal matrices corresponds to adding a weighted (via α) residual connection between each consecutive layer. In this sense, (12) would be a sort of weighted ResNet which, however, does not make use of skip connections. In Section IV-D, comparative experiments between the proposed model with the identity and random orthogonal matrices can be found, with respect to RNN models. Similarly, roaFNN is related to Highway Networks [15], but it circumvents the need of trainable gates at a structural level. Moreover, neither ResNets nor Highway Networks are provided with formal proofs of solving the V/E gradient issue and, even more so, for the case of arbitrarily large depths. On the other hand, the roaFNN model is rooted in solid mathematical bounds regarding the maximum singular value of the IOJ that still holds in the case of infinite depths.

In summary, via backpropagation of the errors, the update rule [see (8) and (9)] modifies the parameters of (12), W_l and b_l, while the matrices O_l automatically keep the gradient norms at a controlled magnitude. *The role of those fixed matrices is essentially to "shuffle" the information encoded in the neuronal activations at each crossed layer (forwardly or backwardly) encouraging dynamical isometry.* This simple trick results to be effective for an FNN to achieve approximate dynamical isometry, i.e., a configuration where the distribution of the singular values of the IOJ remains contained "around" 1 without spreading too far from it. To the best of the author's knowledge, a general method to ensure approximate dynamical isometry in nonlinear NNs has been missing so far. Remarkably, this is accomplished without imposing any constraints on the parameters of the model or relying on specific initialization schemes. Noticeably, a roaFNN is a clear generalization of a standard MLP, which we recover setting $\alpha = 1$, or equivalently setting $\rho = L-1$. Evidently, the bounds of Theorem 1 exponentially diverge to infinity or collapse to zero, in the case of $\rho = L - 1$, for increasingly larger depths.

2) Computational Complexity: Due to the presence of the additional semiorthogonal matrices, both the computation time and memory are doubled in a roaFNN compared to a vanilla FNN. In any case, considering the outstanding improvement of the performance, especially for RNNs (see Section IV), this tradeoff is particularly favorable. Furthermore, in Appendix C, two strategies to bring memory and time consumption of (12) to the same levels of the vanilla FNN are proposed. However, those strategies have not been properly tested in this work; therefore, further investigations are needed.

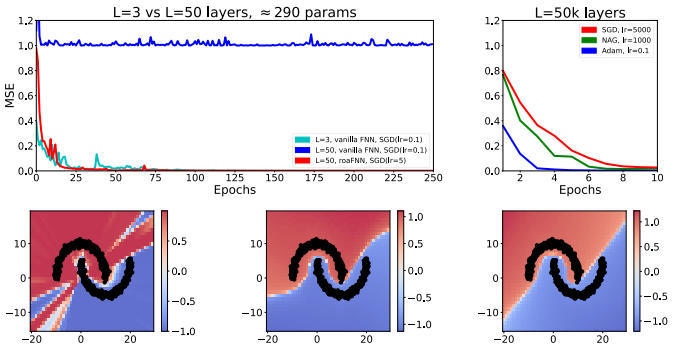


Fig. 2. In cyan, the mse of a shallow vanilla FNN with one hidden layer of dimension 72 (trainable parameters 289) (top left). In blue and red, the mse of, respectively, a deep vanilla FNN and a deep roaFNN, with 48 hidden layers of dimension two (trainable parameters 291). A roaFNN of 50k layers has been trained with three different optimizers (top right). SGD is the stochastic gradient descent (no momentum). NAG is the Nesterov accelerated gradient with 0.99 momentum. Adam is with standard parameters suggested in [46]. The output of the models evaluated in the region $(x_1, x_2) \in [-20, 30] \times [-15, 20]$ (bottom row). From left to right: the shallow $L = 3$ vanilla FNN trained with SGD for 250 epochs, the deep $L = 50$ roaFNN trained with SGD for 250 epochs, and the extremely deep $L = 50k$ roaFNN trained with Adam for ten epochs. The deep $L = 50$ vanilla FNN is not reported since it did not solve the task. *All:* All models have input and hidden layers of dimension two, and an output layer of dimension one. All neurons have tanh activation function. All weights and biases have been initialized according to a normal distribution with zero mean and unitary standard deviation. Batch size is 100. All roaFNN models have $\alpha = \rho(L-1)^{-1}$ with $\rho = 5$. Learning rates (denoted as lr in the figure) of roaFNNs are larger than usual, but they are relatively small considering that they are effectively rescaled by a factor of α in the backpropagation algorithm [see (13)]. Adam is an exception since it autonomously adapts the learning rate.

D. Double-Moon Regression Task

In this section, as a proof of concept, it is shown that we can propagate signals back and forth through an extremely deep FNN. The task is the double-moon regression task [45]. A number of 1000 sample points (x_1, x_2) in the real plane are generated in a double-moon fashion: black dots in the second-row plots of Fig. 2. Points of one moon are labeled with target value -1 and the others as 1. Based on the double-moon training dataset, an FNN is asked to generalize a mapping from $\mathbb{R}^2$ to $\mathbb{R}$. It is a pretty trivial regression task for a shallow vanilla FNN with one hidden layer of 72 neurons. Nevertheless, for a vanilla FNN with the same number of trainable parameters spread over 48 hidden layers ($L = 50$), it turns out impossible. On the contrary, a 50-layer roaFNN converges fast and smoothly to a solution in a few epochs. The same rate of convergence is achieved with a 50 000-layer roaFNN, with no gradient clipping or any other "trick" (e.g., batch normalization, skipping connections, etc.). This corroborates the resilience of the proposed model to V/E gradient issues. For details of the simulations, see the caption of Fig. 2.

Unlike the vanilla FNN, the roaFNN model presents a smooth decision boundary, even for the 50k layers model

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

despite the abnormally large number of parameters to tune (≈300k). The smoothness of the learning curve and the decision boundary for roaFNN are likely due to the presence of the orthogonal matrices in (12) that may have the effect to regularize the landscape of the loss function and stabilize the learning dynamics; note that no explicit regularization has been introduced in any model. Both strategies (I) and (II) proposed in Appendix C have been tested on this task, and they both worked fine (data not reported).

III. Learning Long-Term Dependencies With RNNs

The idea presented for FNNs in Section II can be implemented in any kind of NN model. In this section, we will implement the idea on *vanilla RNNs*. Generally, an RNN processes an input sequence $\{u_k\}$, developing an internal state sequence $\{x_k\}$ and generating an output sequence $\{z_k\}$, by means of the following state-update equations:

$$x_{k+1} = \phi(W_h x_k + b_h + W_i u_{k+1}) \tag{17}$$
$$z_{k+1} = \psi(W_o x_{k+1} + b_o). \tag{18}$$

Typical choices of ψ are the identity function or a softmax function.

Starting with an initial condition x_0 for the internal state, and considering a finite window of evolution of L time steps, we can interpret (17) and (18) as a special FNN driven by the finite input sequence $(u_1, u_2, \ldots, u_L)$. Therefore, we can fictitiously unfold the RNN in an FNN with (internal) input layer x_0 passing through L hidden layers that share the same weights and biases, W_h and b_h, and driven by an (external) input sequence $(u_1, u_2, \ldots, u_L)$ via further shared parameters W_i. Hence, we can use the backpropagation algorithm to update the parameters of (17) and (18). This is called the backpropagation through-time (BPTT) algorithm. As a consequence, such an RNN must be able to learn dependencies in the input sequence for up to L time steps in the past to function correctly. However, when it comes to learn long-term dependencies, a vanilla RNN model of (17) and (18) suffers terribly from the V/E gradient issue. For example, for a vanilla RNN, the IOJ reads $\prod_{l=L-1}^{1} \mathrm{diag}(d_l) W_h$, whose norm can be upper bounded with $\|W_h\|^{L-1}$, whenever the activation function assumes derivative values in $[0, 1]$. Hence, for large values of L, if $\|W_h\| < 1$, we observe vanishing gradients. Note that even with orthogonal weights, i.e., $\|W_h\| = 1$, the gradient might still vanish due to the matrices $\mathrm{diag}(d_l)$.

A simple way to enhance the computational power of a vanilla RNN is suggested by the arguments discussed in Section II. The proposed RNN model (see Fig. 3) reads

$$x_{k+1} = \alpha\phi(W_h x_k + b_h + W_i u_{k+1}) + (1-\alpha) O x_k \tag{19}$$
$$z_{k+1} = \psi(W_o x_{k+1} + b_o) \tag{20}$$

where O is a random orthogonal matrix. In the remainder of this article, (19) and (20) will be referred to as roaRNN, which stands for random orthogonal additive RNN. In summary, we simply add a random orthogonal matrix shared between "all the hidden layers" of an RNN. Note that in the limit of $\alpha = 1$, we recover the vanilla RNN model.

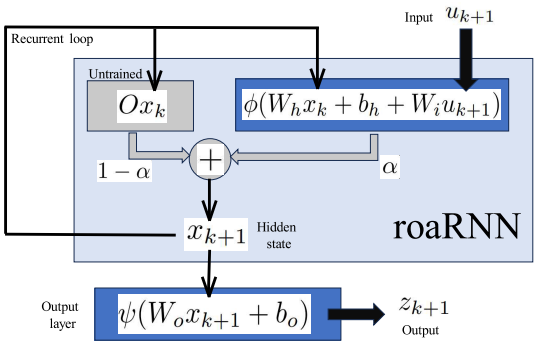


Fig. 3. Scheme of roaRNN, defined in (19) and (20). Gray parts are untrained, and blue parts are adapted with SGD via BPTT.

Remark 6: The orthogonal matrix O characterizes the internal redistribution of the information into the network. The matrix $(1-\alpha)O$ has eigenvalues inside the unit disk, close to the unit circle for smaller values of α. Such a close-to-orthogonal matrix represents the slightly dissipative internal dynamics of the recurrent network. Notably, an $0 < \alpha < 1$ ensures that the dynamical system defined by (19) evolves inside the hypercube $[-R, R]^{N_h}$, as long as the activation function ϕ is bounded in $(-R, R)$, nondecreasing, and upper semicontinuous (see [47, Prop B.1] for a proof of this fact).

Parameters of (19) and (20) are W_h, b_h, W_i, W_o, and b_o. We want to adapt the parameters of the model proportionally to the derivative of the loss function with respect to them.

Gradients of "hidden" parameters, i.e., W_h, b_h, and W_i, need special attention since weights are shared across temporal layers. For instance, the gradient with respect to the hidden-to-hidden weights takes the form

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{s=0}^{L-2} \frac{\partial \mathcal{L}}{\partial W_{h,s}} \tag{21}$$

where the same hidden-to-hidden matrix W_h has been fictitiously labeled as $W_{h,s}$, as if they were different matrices at each temporal layer. For each $s = 0, \ldots, L-2$, it holds

$$\frac{\partial \mathcal{L}}{\partial W_{h,s}} = \alpha\left[\mathrm{diag}\left(\phi'(y_s)\right)\mathcal{R}_s(E)\right] \otimes x_s$$
$$\mathcal{R}_s(E) = \left(\frac{\partial x_L}{\partial x_{s+1}}\right)^{\mathrm{T}} \left(\mathrm{diag}\left(\psi'(y_L)\right) W_o\right)^{\mathrm{T}} E$$
$$\frac{\partial x_L}{\partial x_{s+1}} = \frac{\partial x_L}{\partial x_{L-1}} \frac{\partial x_{L-1}}{\partial x_{L-2}} \cdots \frac{\partial x_{s+2}}{\partial x_{s+1}}$$

where E is the error vector, $y_l = W_h x_l + b_h + W_i u_{l+1}$, and

$$\frac{\partial x_{l+1}}{\partial x_l} = \alpha\, \mathrm{diag}\left(\phi'(y_l)\right) W_h + (1-\alpha) O. \tag{22}$$

The analysis of roaRNN gradients is much easier than roaFNNs since we deal with a square IOJ matrix. In fact, we can bound completely the distribution of singular values of the IOJ, as stated in the following theorem.

Theorem 2: Let us consider a roaRNN with an activation function ϕ such that its derivative assumes values in $[0, r]$. Let us be given any initial state x_0, driven by the input sequence $(u_1, \ldots, u_L)$. Therefore, via (19), it is well-defined

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

the input driven trajectory $\{(x_0, u_1), \ldots, (x_{L-1}, u_L)\}$. Denote $\sigma = \|W_h\|$. Assume $\alpha = \rho(L-1)^{-1}$, such that $\rho < ((L-1/))(1+r\sigma)$, then the whole set of singular values of the IOJ matrix $(\partial x_L/\partial x_1)$ is contained in the L-independent interval

$$\big[\exp(-\rho(1+r\sigma)), \exp(\rho(r\sigma-1))\big].$$

The upper bound on the maximum singular value is a straightforward application of (15) of Theorem 1 for the case of roaRNN. The proof of the lower bound on the *minimum* singular value can be found in Appendix B.

Putting all together in (21), we get the expression

$$\frac{\partial \mathcal{L}}{\partial W_h} = \alpha \sum_{s=0}^{L-2} \big[\operatorname{diag}\big(\phi'(y_s)\big)\mathcal{R}_s(E)\big] \otimes x_s. \tag{23}$$

The gradients with respect to the other hidden parameters b_h and W_i have the same form of (23), but, respectively, omitting the outer products with the vectors x_s and replacing the outer products with the vectors x_s with the vectors u_{s+1}.

Remark 7: The matrix (23) is a sum of $L-1$ terms; each one depending on a different depths of the internal representation $\{x_k\}$ of the listened input sequence, and the input sequence itself. Because of the parametrization $\alpha = \rho(L-1)^{-1}$, the sum (23) is effectively a sort of average of all the contributions from the past L time steps.

Remark 8: Note that Theorem 2 still holds with a roaRNN where the orthogonal matrix is the identity matrix. Interestingly, such a model has been already proposed in the deep learning literature with the name of FastRNN [26], and in the reservoir computing literature with the name of leaky-integrator ESN [48]. Remarkably, theoretical results on convergence and generalization error bounds exist for FastRNN [26, Ths. 3.1–3.2] whose proofs automatically hold for the roaRNN architecture. However, experiments suggest that the identity does not seem to provide performance as good as a random orthogonal matrix (see Section IV-D). The conjecture is that the characteristic annular distribution of the eigenvalues of the Jacobians (22) may play a key role. In fact, the eigenvalues of $\alpha \operatorname{diag}(\phi'(y_l))W_h + (1-\alpha)O$ can be tuned arbitrarily close to the complex unitary circle as α approaches zero, regardless of the matrix W_h. On the other hand, replacing O with the identity matrix makes the eigenvalues converge in the complex plane to the point $(1, 0)$ as $\alpha \to 0$. This might negatively affect the expressive power of the model. Further comparative experiments of identity versus random orthogonal can be found in [49], [50], and [51], from the point of view of reservoir computing.

IV. Experiments on RNNs

In this section, the roaRNN model is benchmarked on three tasks specifically devised to be challenging for recurrent models to learn long-term dependencies [21]: 1) copying memory (MemCop); 2) adding problem (AddProb); and 3) permuted sequential MNIST (psMNIST).

The roaRNN model is compared against vanilla RNN[3] with ReLU activation function, and LSTM-RNN, which is by far the most popular RNN model used to learn long-term dependencies.

1) *RoaRNN Setting:* For all the considered tasks, all parameters W_h, b_h, W_i, W_o, and b_o are initialized according to a normal distribution with zero mean and unitary standard deviation. The activation function ϕ is ReLU. The readout function ψ in (20) is chosen as the identity, that is, a linear output layer. The learning rate, after random trials in the range $[10^{-4}, 10]$, has been set to 0.5, and kept constant throughout the learning for the MemCop and AddProp tasks, while for the psMNIST, it has been set to 0.1 in the first ten epochs, and then to 0.01 for the next ten epochs. The hyperparameter ρ is optimized on each task on the grid of values
$$\left\{\frac{1}{300}, \frac{1}{200}, \frac{1}{100}, \frac{1}{50}, \frac{1}{10}, \frac{1}{2}, 1, 2, 3, 5, 10, 50\right\}.$$
2) *Vanilla RNN Setting:* The hidden-to-hidden matrix is initialized as an orthogonal matrix. All the other parameters are initialized according to a uniform distribution in $(-\sqrt{k}, \sqrt{k})$, where $k = N_h^{-1}$ is the reciprocal of the hidden size. The activation is ReLU. The readout function ψ in (20) is chosen as the identity, that is, a linear output layer. The learning rate for the MemCop and AddProb tasks, after random trials in the range $[10^{-6}, 1]$, has been set to 10^{-4}, and kept constant throughout the learning.
3) *LSTM Setting:* All the four hidden-to-hidden matrices are initialized as orthogonal matrices. All the other parameters are initialized according to a uniform distribution in $(-\sqrt{k}, \sqrt{k})$, where $k = N_h^{-1}$ is the reciprocal of the hidden size. As usual, the activation is tanh, while gates are implemented with sigmoids. The readout function ψ in (20) is chosen as the identity, which is a linear output layer. The learning rate for the MemCop and AddProb tasks, after random trials in the range $[10^{-6}, 1]$, has been set to $5 \cdot 10^{-3}$ and kept constant throughout the learning.
4) *Optimizer:* All models are trained with the Adam optimizer, according to the standard setting proposed in [46]. No regularization, no gradient clipping, and no batch normalization were used in any model.
5) *Hardware:* CPU is an 11th Gen Intel Core i7-11370H. GPU is a 4-GB NVIDIA GeForce GTX 1650.
6) *Software:* Experiments have been run with PyTorch 1.10.1, and cudatoolkit 10.2.89.
7) *Trials:* To wash out unlucky initializations from the comparison, five randomly generated seeds between 0 and 2^{30} have been used to run each model. The plots regarding the MemCop and AddProb tasks refer to the best results obtained by each model out of the five trials.

[3]The vanilla RNN model used here is a slight variation of (17) where there are two bias vectors to learn. This is the default PyTorch implementation of an RNN at present.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

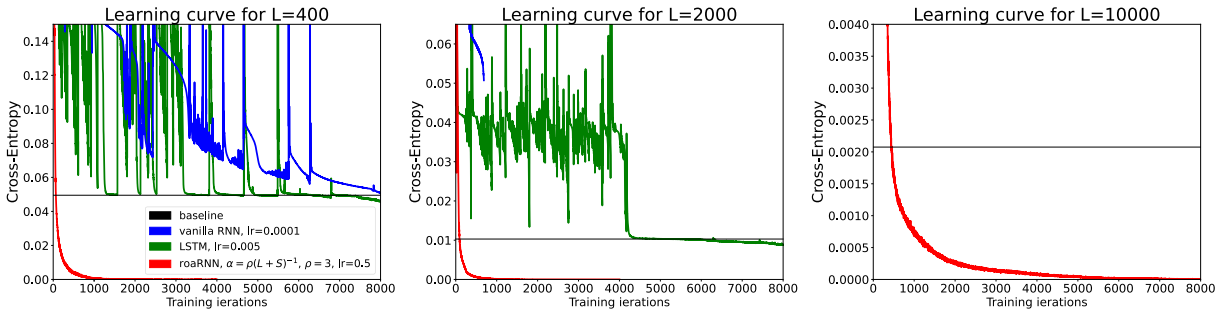


Fig. 4. Copying memory task. There are $S = 10$ symbols to recall after a time lag of L time steps. Cross-entropy loss of vanilla RNN (blue), LSTM (green), and roaRNN (red), for three cases of time lags $L = 400, 2000$, and 10 000. In all three cases, roaRNN is set with $\alpha = \rho(L + S)^{-1}$, with a $\rho = 3$. In the legend, lr denotes the learning rate.

8) *Results:* Remarkably, roaRNN reaches the state-of-the-art performance on all tasks, despite the minimal change from a vanilla RNN model.[4]

A. Copying Memory Task

1) Task: Let us be given an alphabet of eight symbols that we can codify as $\mathcal{A} = \{1, 2, 3, 4, 5, 6, 7, 8\}$. The input is a sequence composed of three concatenated blocks of lengths (S, L, S). The first S time steps are symbols sampled i.i.d. uniformly at random from the alphabet $\mathcal{A}$. Then, there are L time steps of blank characters that we can codify as 0. Finally, there is a start character codified as nine followed by $S - 1$ blank characters. The RNN must generate an output sequence of the same length as the input, where the first $S + L$ outputs are blank characters, and the last block of S time steps must match the first block of ordered symbols of the input sequence. The length of the block of symbols to recall is $S = 10$, as usually set in the literature [21], [22], [23]. The following is an example of input–output pairs with $L = 12$ and $S = 4$, where the blank character 0 has been replaced with $-$, and the start character 9 with :, for easier visualization

Input 5 5 6 1 $- - - - - - - - - - - -$: $- - -$
Output $- - - - - - - - - - - - - - - -$ 5 5 6 1.

In the simulations run, time lags of $L = 400, 2000$, and 10 000 and time steps have been considered. Batches of size 128 are randomly generated on the fly as training proceeds. The loss function is the cross-entropy. The baseline to beat for this task is a cross-entropy value of $S \log(8)/(L + 2S)$, corresponding to output $L + S$ blank characters followed by S symbols at random.

2) Optimal ρ Setting: Note that different values of L give different optimal values of ρ. However, the setting of $\alpha = \rho(L+S)^{-1}$, with a $\rho = 3$, behaves quite well for all the three considered cases of $L = 400, 2000$, and 10 000. ExpRNN [23] and scoRNN [24] achieve the best performance in the literature for the MemCop task. They use 190 units in their simulations, so the same number has been used here for all three models, vanillaRNN, LSTM, and roaRNN, for the sake of comparison. Note, however, that an LSTM of 190 hidden units has a considerably larger number of trainable parameters than a vanilla RNN and roaRNN of the same size. Nonetheless, as clear from Fig. 4, roaRNN abundantly outperforms LSTM. As already reported in the literature, LSTMs truly struggle to solve the MemCop task even with relatively short time lags of $L = 400$, albeit obtaining slightly better performance than vanilla RNNs. On the contrary, roaRNNs are characterized by a smooth learning curve, and a fast convergence regardless of the length of the time lag L. Even in the extreme case of $L = 10\,000$, the roaRNN model beats the baseline within the first 500 iterations. In the cases of $L = 400$ and $L = 2000$, the roaRNN training has been stopped at the 4000th iteration since it reached 100% accuracy on the vast majority of the training steps after the 2000th. Running for longer sometimes gives rise to instabilities. For the case of $L = 2000$, all five trials of vanilla RNNs gave NaN values, testifying a V/E gradient issue. The ones plotted in Fig. 4 are those that got NaN values the latest. All the simulations for the MemCop tasks have been run with a batch size of 128, as in [23].

B. Adding Problem Task

1) Task: The RNN is driven with two input sequences, $u_k = (a_k, b_k)$, of the same length T. The first sequence a_k assumes values according to a uniform distribution of real values in $[0, 1]$. The second sequence b_k assumes all zeros apart from two time steps, for $k = i_1$ and $k = i_2$, which are marked with the value 1. The markers i_1 and i_2 are (uniformly) randomly selected in the first half of the sequence and the second half of the sequence, respectively, i.e., at positions $1 \leq i_1 \leq \lfloor (T/2) \rfloor < i_2 \leq T$. The RNN must "recognize" the two real numbers corresponding to the positions marked with 1, ignoring the other $T - 2$ time steps of uniformly distributed noise, and produce in output the sum of them, i.e., $a_{i_1} + a_{i_2}$. The following is an example of input–output pairs with $T = 8$ where the decimal parts are truncated to 10^{-2} for easier visualization:

Input $\begin{bmatrix} 0.71 & 0.14 & 0.64 & 0.77 & 0.54 & 0.26 & 0.88 & 0.15 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$
Output $0.14 + 0.26$.

In the simulations run, lengths of $T = 200, 1000$, and 5000 and time steps have been considered. The loss function

[4]Different combinations of initializations, e.g., all parameters uniformly random in $(-1, 1)$, optimizers, e.g., SGD, and activation functions, e.g., tanh, work good as well. However, hyperparameters such as ρ and the learning rate should be changed accordingly for optimal performance.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

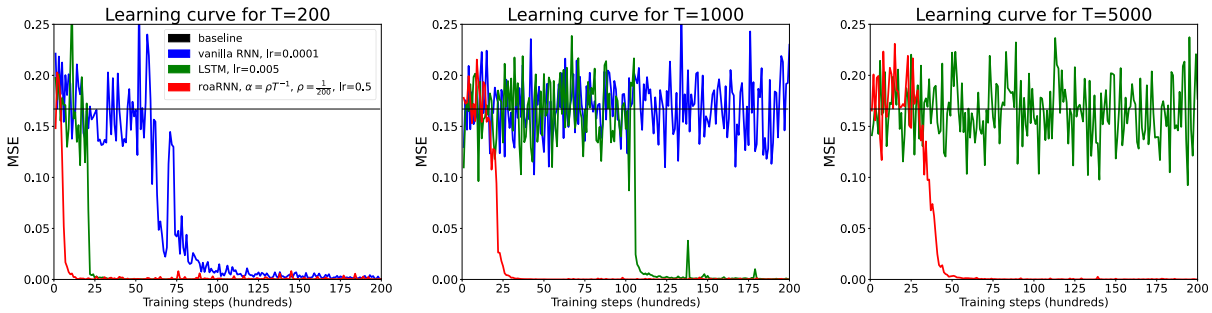


Fig. 5. Adding problem task. The mse of vanilla RNN (blue), LSTM (green), and roaRNN (red), for the three cases of sequence lengths $T = 200$, 1000, and 5000. In all three cases, roaRNN is set with $\alpha = \rho T^{-1}$, with a $\rho = (1/200)$. In the legend, lr denotes the learning rate.

TABLE I

MINIMUM MSES (OF FIG. 5) ON THE ADDPROB TASK

	$T = 200$	$T = 1000$	$T = 5000$
Vanilla RNN	3.6×10^{-4}	–	–
LSTM	1.1×10^{-5}	2.3×10^{-4}	–
roaRNN	1.6×10^{-5}	2.3×10^{-6}	2.5×10^{-6}

for this task is the mse. The baseline mse value to beat is 0.167, i.e., the variance of the sum of two independent uniform distributions in [0, 1], corresponding to predict the output value 1 regardless of the input sequence. Batches of size 50 are randomly generated as training proceeds. At each 100 training steps, the model is evaluated on a randomly generated batch on the fly and plotted in Fig. 5.

2) Optimal ρ Setting: Again, different values of T give different optimal values of ρ. However, the setting of $\alpha = \rho T^{-1}$, with a $\rho = (1/200)$, behaves quite well for all the three considered cases of $T = 200$, 1000, and 5000.

CoRNN [31] and IndRNN [27] achieve the best performance in the literature for the AddProb task. They use 128 hidden units, so the same size has been implemented here for all of the three vanilla RNN, LSTM, and roaRNN. The AddProb turned out a less challenging task for both LSTMs and vanilla RNN. Their results are better than those usually reported in [21], [52], [53], and [54], probably due to the orthogonal initialization scheme implemented here. However, for long sequences of $T = 5000$, LSTM struggles to solve the task in a reasonable time, and vanilla RNN already fails with $T = 1000$. Contrariwise, roaRNN converges fast, within the first 50 hundred training steps in all the considered cases. The loss curves in Fig. 5 are plotted over hundreds of training iterations to facilitate comparison with results from existing literature, among which [27] and [31]. In Table I, the minimum mses reached by the curves in Fig. 5 are reported, for the cases where the baseline is consistently beaten.

C. Permuted Sequential MNIST Task

1) Task: The training dataset is the whole standard MNIST set of 60k samples. The test dataset is the whole standard MNIST set of 10k samples. Training and test input samples are flattened (from left to right and top to bottom) to be sequences of $T = 784$ pixels. Then, all the sequences are rearranged according to a given permutation (see Fig. 6).[5]

[5]The permutation has been taken from https://www.nengo.ai/nengo-dl/examples/lmu.html, in order to faithfully compare the results obtained in this work with the results of LMU [32] and NRU [33].

TABLE II

TEST ACCURACY OF SEVERAL DEEP LEARNING MODELS ON PSMNIST

Model	Trainable parameters	Test accuracy
Vanilla RNN [35]	≈68k	71.6%
LSTM [24]	≈270k	92.9%
GRU [35]	≈200k	94.1%
Dilated CNN [35]	≈46k	96.7%
FC uRNN [22]	≈270k	94.1%
BN LSTM [55]	–	95.4%
expRNN [23]	≈137k	96.6%
Lipschitz RNN [30]	≈ 34k	96.3%
res-IndRNN [52]	–	97.02%
dense-IndRNN [52]	–	97.2%
NRU [33]	≈165k	95.38%
LMU [32]	≈102k	97.15%
coRNN [31]	≈134k	97.3%
TARNN [56]	≈68k	97.13%
Delayed Memory Unit [57]	≈ 49k	96.39%
Deep Memory Update [58]	≈ 34k	93.4%
roaRNN	≈ 34k	97.24%
roaRNN	≈ 69k	97.88%
roaRNN	≈270k	98.25%

2) Optimal ρ Setting: Among the grid of ρ values of

$$\left\{\frac{1}{300}, \frac{1}{200}, \frac{1}{100}, \frac{1}{50}, \frac{1}{10}, \frac{1}{2}, 1, 2, 3, 5, 10, 50\right\}$$

the best results have been found with $\rho = 1/2$, i.e., setting the hyperparameter $\alpha = 1/2T$.

Table II reports the best test accuracy values on the psMNIST task reached by a wide plethora of NN models ranging from stacked multilayer architectures (as dense-IndRNN and Dilated CNN), gated architectures (as GRU and LSTM), models provided with memory cells (as NRU and LMU), unitary learning models (as FC uRNN and expRNN), continuous-time ODE models (as Lipschitz RNN), second-order models (as coRNN), and other models specifically devised to tackle the V/E gradient issue of RNNs. Each result reported in Table II is accompanied by the reference in the literature where it is found. For some of them, the number of trainable parameters is not available. However, note that these results are not fully fairly comparable to each other due to different training methods (optimizer, batch size, etc.), and permutation used for training the models.

RoaRNNs with hidden sizes of 178, 256, and 512 have been trained for a number of five trials each. The accuracy on the whole test dataset has been computed at the end of each epoch and is plotted in Fig. 7 for each trial. The darker the blue color, the larger the size of the recurrent layer. In all cases, the output layer consists of ten neurons, one for each digit to

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.



Fig. 6. First training sample of the MNIST dataset is a 28×28 pixel image representing a handwritten digit five (left). The image representing the digit five is flattened in a $784 = 28^2$ sequence starting from the top-left corner, proceeding from left to right and top to bottom, until the bottom-right corner (top right). The sequence is plotted in eight rows for better visualization. The resulting sequence is then permuted according to a predefined permutation (bottom right).

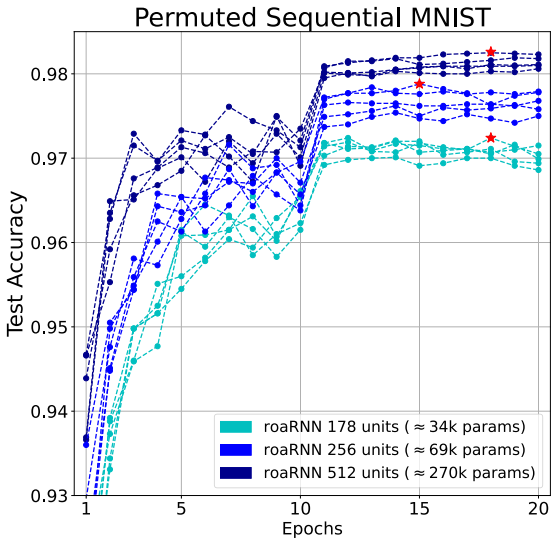


Fig. 7. Test accuracy on psMNIST for ten different initializations of roaRNN of three different sizes. Red stars mark the highest test accuracy reached in 20 epochs for each one of the three sizes.

classify between $\{0, 1, \ldots, 9\}$. The loss function is the cross-entropy. Training data are shuffled at the end of each epoch. Batch size is 100. The learning rate is 0.1 for the first ten epochs, and then 0.01 from the 11th epoch to the 20th.

Remarkably, most of the NN architectures reported in Table II need to be trained for several dozens (if not hundreds) of epochs to reach good performance. On the contrary, roaRNN reaches test-accuracy values above 97% after just ten epochs in all cases, outperforming rapidly the vast majority of alternative deep learning architectures.

D. Ablation Study: Identity Versus Random Orthogonal Matrices

In this section, the performance of the RNN model of (19) and (20) with random orthogonal O (roaRNN) is compared against the peculiar case of the identity matrix $O = I$ (eyeRNN) (see Fig. 8). Specifically for both models, five simulations based on five different initializations of parameters have been run, for each one of the three selected tasks (MemCop, AddProb, and psMNIST). In particular, the roaRNN's and eyeRNN's parameters are made to exactly coincide when learning starts,[6] i.e., differing exclusively between each other in the choice of the matrix O. Preliminary trials on eyeRNN, in all of the three tasks considered in this section, revealed that a good learning rate for eyeRNN is generally ten times smaller than the one for roaRNN. Both models have been run with 128 hidden units, Adam optimizer, and a standard choice of $\rho = 1$, in all three tasks. In the AddProb task, eyeRNN performed significantly better with tanh activation rather than ReLU. Therefore, for this task, the comparison has been done between tanh-eyeRNN and tanh-roaRNN. While MemCop and psMNIST have been run with ReLU. For the psMNIST task, training was stopped at the 20th epoch, albeit eyeRNN models were still learning since it was enough to prove the disparity between the two variants.

Note that the bounds on the distribution of singular values of the IOJ provided in Theorem 2 hold for any orthogonal matrix and for the identity matrix as well. Nevertheless, as suggested by the results in Fig. 8, the identity does not give results as good as a random orthogonal matrix does. *The conjecture is that the particular annular distribution of eigenvalues of the roaRNN gradient helps the learning dynamics to reach a desired eigenspectrum configuration for the task at hand. While for the case of the identity matrix, the eigenspectrum of the RNN gradient is not heterogeneously spread over the unitary circle, and thus, it might need a greater learning effort to adapt itself to certain tasks.* An extended analysis would be ideal to establish which are the important features that an orthogonal matrix must possess to facilitate further the learning process.

V. Future Works

The gradient dynamics of the proposed model have been proved in this article to be stable at the level of each parameter's update, but a theoretical investigation of the global learning dynamics has been neglected. The experiments run on both roaFNN and roaRNN indicate that a favorable side effect of exploiting random orthogonal "filters" might be to effectively regularize the learning dynamics. How these orthogonal matrices alter the geometry of the loss landscape is a compelling research question. Moreover, further investigations are needed to enlighten the role that specific structured orthogonal matrices can have on the proposed model, as suggested by the difference in performance observed between the case of the identity matrix versus random orthogonal matrices. Besides, an implicit assumption of the proposed model is the existence of an optimum scalar value for the hyperparameter ρ. However, there might be complex tasks where the nonlinear dependence in ρ is intrinsically nonconvex, resulting in many local optimal values; or cases where it might be helpful to retrieve information in the past activations at different time lags for different spatial components. This makes interesting to experiment an adapting multidimensional

[6]This is ensured by using the same set of five seeds for each task.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

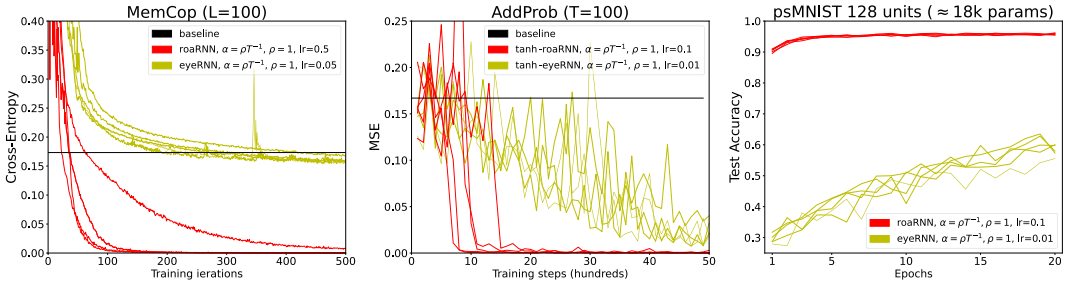


Fig. 8. Comparison between the model of (19) and (20), where the orthogonal matrix O is the identity matrix (eyeRNN) and a randomly generated one (roaRNN).

vector ρ in the proposed model. Since the vector ρ can be linked to the timescales of the neurons in a continuous-time RNN [48], the derivation of a continuous-time version of the proposed roaRNN model represents an interesting direction for future research. Such a continuous-time roaRNN could provide greater flexibility, making it especially suited for tasks requiring high temporal resolution and continuity, e.g., modeling dynamical systems or processing irregularly sampled time series.

A further promising research direction is replacing the MLP blocks in Transformer architectures with the proposed roaFNN model. This adaptation could potentially improve the capacity of Transformer backbones in large language models to effectively leverage extreme depths, possibly enabling more expressive representations and improved performance. Such an extension would also provide valuable insights into the interplay between depth scalability and model expressivity within modern NN architectures.

Finally, although many of the proposed solutions in the literature so far to combat V/E gradients (e.g., skipping connections, batch normalization, trainable gates, etc.) could still be implemented on the architecture proposed here, it should be a matter of research to establish whether the combination of these techniques comports more advantages than unreasonable complexity in a model, which is already afflicted by interpretability and explainability issues.

VI. Conclusion

In this article, a new technique to pursue approximate dynamical isometry in generic nonlinear FNNs is proposed. Loosely speaking, it suffices to add fixed random semiorthogonal matrices between layers, suitably weighted via convex combination with the nonlinear trainable residual component, to consent the signals to flow back and forth along the NN. The same intuition leads to a slight variation of the vanilla RNN model, called roaRNN, which can be trained via BPTT to learn arbitrarily long-term dependencies in the input. The performance of a roaRNN is exceptional considering its ease of implementation and at the cost of just one more hyperparameter to tune, i.e., ρ.

The simplicity of the proposed model makes it amenable to a rigorous mathematical analysis of the gradient update dynamics and yields an intelligible solution to a problem that afflicted NN models from the early nineties.

Appendix A
Maximum Singular Value's Bounds of roaFNN

Recall the roaFNN's equation

$$x_{l+1} = \alpha\phi(W_l x_l + b_l) + (1-\alpha)O_l x_l, \quad l = 0, 1, \ldots, L-1.$$

Recall that, for $l = 0, \ldots, L-1$, matrices W_l have dimensions $N_{l+1} \times N_l$, and O_l are random semiorthogonal matrices of the same dimensions. Recall also that $d_l = \phi'(W_l x_l + b_l)$ are vectors that vary in $[0, r]^{N_{l+1}}$, and denote $\sigma_l := \|W_l\|$, and $\sigma_{\max} := \max_l \sigma_l$, where $\|\cdot\|$ denotes the matrix norm induced by the Euclidean norm of vectors, i.e., the maximum singular value. Let us define matrices

$$P_l := P_{l+1}\big[\alpha \operatorname{diag}(d_l) W_l + (1-\alpha)O_l\big]$$

starting with P_L as the identity matrix of dimension N_L. Thus, by definition, P_1 is the IOJ of the roaFNN.

Lemma 1: For each $l = 1, \ldots, L-1$, the following inequality holds $\|P_l\| \le [1 + \alpha(r\sigma_l - 1)]\|P_{l+1}\|$.

Proof: It follows from the triangle inequality applied on the matrix P_l. In fact, $\|P_l\| = \|\alpha P_{l+1} \operatorname{diag}(d_l) W_l + (1-\alpha)P_{l+1}O_l\| \le \alpha\|P_{l+1}\operatorname{diag}(d_l)W_l\| + (1-\alpha)\|P_{l+1}O_l\| \le \alpha r\sigma_l\|P_{l+1}\| + (1-\alpha)\|P_{l+1}\| = [1 + \alpha(r\sigma_l - 1)]\|P_{l+1}\|$. The second inequality follows from the fact that $d_l \in [0, r]^{N_{l+1}}$, the definition of $\sigma_l = \|W_l\|$, and the isometric property of semiorthogonal matrices that imply $\|O_l\| = 1$. □

Lemma 2: Let be given a matrix P of dimension $h \times m$, and a semiorthogonal matrix O of dimension $m \times n$. If $m \le n$, then the following holds $\|PO\| = \|P\|$.

Proof: By sub-multiplicativity, the isometric property of O holds $\|PO\| \le \|P\|\|O\| \le \|P\|$. By contradiction, assume that $\|PO\| < \|P\|$. This must be true for the transpose of these matrices as well. Hence, we have that $\|O^{\mathrm{T}}P^{\mathrm{T}}\| < \|P^{\mathrm{T}}\|$. Let us denote $v_0, v_1 \in \mathbb{R}^h$, the two unitary vectors that maximize, respectively, the linear operators $O^{\mathrm{T}}P^{\mathrm{T}}$ and P^{T}. By assumption of $\|O^{\mathrm{T}}P^{\mathrm{T}}\| < \|P^{\mathrm{T}}\|$, we have that

$$\|O^{\mathrm{T}}P^{\mathrm{T}}v_0\| < \|P^{\mathrm{T}}v_1\|. \tag{24}$$

By hypothesis, O is a semiorthogonal matrix of dimension $m \times n$ with $m \le n$. This implies that $OO^{\mathrm{T}} = I_m$, which, in turn, implies that $\|O^{\mathrm{T}}w\| = \|w\|$, for all $w \in \mathbb{R}^m$. In fact, $\|O^{\mathrm{T}}w\|^2 = (O^{\mathrm{T}}w)^{\mathrm{T}}O^{\mathrm{T}}w = w^{\mathrm{T}}OO^{\mathrm{T}}w = w^{\mathrm{T}}w = \|w\|^2$. Therefore, for the vector $w_1 = P^{\mathrm{T}}v_1$, we get

$$\big\|P^{\mathrm{T}}v_1\big\| = \|w_1\| = \big\|O^{\mathrm{T}}w_1\big\| = \big\|O^{\mathrm{T}}P^{\mathrm{T}}v_1\big\| \le \big\|O^{\mathrm{T}}P^{\mathrm{T}}v_0\big\| \tag{25}$$

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

where the last inequality holds since v_0 maximizes the linear operator $O^{\mathrm{T}}P^{\mathrm{T}}$. The inequality (25) contradicts (24). □

Lemma 3: Let us assume a roaFNN model with nonincreasing sizes of layers, i.e., assume $N_0 \geq N_1 \geq \ldots, \geq N_L$. If $\alpha < (1/(1+r\sigma_{\max}))$, then it holds $\|P_l\| \geq [1-\alpha(1+r\sigma_l)]\|P_{l+1}\|$, for $l = 1, \ldots, L-1$.

Proof: The hypothesis that $\alpha < (1/(1+r\sigma_{\max}))$ implies that $r\sigma_l\alpha < 1-\alpha$, for all $l = 0, \ldots, L-1$. Thus, it follows that:

$$\alpha\|P_{l+1}\operatorname{diag}(d_l)W_l\| \leq \alpha\sigma_l r\|P_{l+1}\| \leq (1-\alpha)\|P_{l+1}\|. \quad (26)$$

Now, by hypothesis $N_{l+1} \leq N_l$, so Lemma 2 ensures that $\|P_{l+1}\| = \|P_{l+1}O_l\|$. Hence, from (26), we obtain

$$\alpha\|P_{l+1}\operatorname{diag}(d_l)W_l\| \leq (1-\alpha)\|P_{l+1}O_l\|. \quad (27)$$

Therefore, (27) allows us to write the reverse triangle inequality in the following form $(1-\alpha)\|P_{l+1}O_l\| - \alpha\|P_{l+1}\operatorname{diag}(d_l)W_l\| \leq \|\alpha P_{l+1}\operatorname{diag}(d_l)W_l + (1-\alpha)P_{l+1}O_l\| = \|P_l\|$, which can be rewritten, according to Lemma 2, as $(1-\alpha)\|P_{l+1}\| - \alpha\|P_{l+1}\operatorname{diag}(d_l)W_l\| \leq \|P_l\|$. The left term of the last inequality can be bounded from the bottom using the first inequality in (26) obtaining

$$(1-\alpha)\|P_{l+1}\| - \alpha\sigma_l r\|P_{l+1}\| \leq \|P_l\| \quad (28)$$

which can be rearranged as $[1-\alpha(1+r\sigma_l)]\|P_{l+1}\| \leq \|P_l\|$, and that is the thesis. □

Theorem 3: Let us consider an roaFNN model with an activation function ϕ such that its derivative assumes values in $[0, r]$. Let us denote $\sigma_{\max} := \max_l\|W_l\|$. If we parametrize $\alpha = \rho(L-1)^{-1}$, with a positive real-valued ρ, then the maximum singular value of the IOJ of the roaFNN admits the following L-independent upper bound $\|(\partial x_L/\partial x_1)\| \leq \exp(\rho(r\sigma_{\max}-1))$. Moreover, if $\rho < ((L-1)/(1+r\sigma_{\max}))$, and assuming nonincreasing sizes of layers, then it holds the following L-independent lower bound $\exp(-\rho(1+r\sigma_{\max})) \leq \|(\partial x_L/\partial x_1)\|$.

Proof: By definition, $P_1 = [\alpha\operatorname{diag}(d_{L-1})W_{L-1} + (1-\alpha)O_{L-1}]\cdots[\alpha\operatorname{diag}(d_1)W_1 + (1-\alpha)O_1] = (\partial x_L/\partial x_1)$ is the IOJ of the roaFNN model.

Upper Bound: Lemma 1 implies that $\|P_1\| \leq \prod_{l=1}^{L-1} 1 + \alpha(r\sigma_l - 1)$, which can be upper-bounded using the definition of $\sigma_{\max} = \max_l\sigma_l$ by the term $[1+\alpha(r\sigma_{\max}-1)]^{L-1}$. Now, with the parametrization $\alpha = \rho(L-1)^{-1}$, it holds $\|(\partial x_L/\partial x_1)\| \leq [1+((\rho(r\sigma_{\max}-1))/(L-1))]^{L-1} \leq \mathrm{e}^{\rho(r\sigma_{\max}-1)}$, where the last inequality holds since $(1+(A/x))^x < \mathrm{e}^A$ for all $x > 0$.

Lower Bound: Lemma 3 implies that $\|P_1\| \geq \prod_{l=1}^{L-1} 1 - \alpha(1+r\sigma_l)$, which can be lower-bounded using the definition of $\sigma_{\max} = \max_l\sigma_l$ by the term $[1-\alpha(1+r\sigma_{\max})]^{L-1}$. Now, with the parametrization $\alpha = \rho(L-1)^{-1}$, it holds

$$\left\|\frac{\partial x_L}{\partial x_1}\right\| \geq \left[1-\frac{\rho(1+r\sigma_{\max})}{L-1}\right]^{L-1} \geq \left[1-\frac{\rho(1+r\sigma_{\max})}{L-1}\right]^{L} \geq \mathrm{e}^{-\rho(1+r\sigma_{\max})}$$

where the second inequality holds because of the hypothesis that $\rho < ((L-1)/(1+r\sigma_{\max}))$, and the last inequality holds because $(1+(A/x))^{x+1} > \mathrm{e}^A$ for all $x > 0$. □

Appendix B
Minimum Singular Value's Lower Bound of roaRNN

In the case of roaRNN, we deal with square matrices. If they are invertible, then we can easily find a lower bound for the minimum singular value of the IOJ matrix J, just exploiting the fact that $\sigma_{\min}(J) = (1/\|J^{-1}\|)$.

Lemma 4: Let us be given two square matrices $A, B \in \mathbb{R}^{N\times N}$ such that A is invertible and $A^{-1}B$ has a spectral radius less than 1. Then, it holds the following relation $(A-B)^{-1} = [\sum_{k=0}^{\infty}(A^{-1}B)^k]A^{-1}$.

Proof: The formal geometric series $\sum_{k=0}^{\infty}C^k = (I-C)^{-1}$ holds for all squared matrices C having spectral radius less than 1. Therefore, by hypothesis, it holds $\sum_{k=0}^{\infty}(A^{-1}B)^k = (I-A^{-1}B)^{-1}$. Therefore, multiplying on the right both terms of the above equation by the matrix A^{-1}, we get $[\sum_{k=0}^{\infty}(A^{-1}B)^k]A^{-1} = (I-A^{-1}B)^{-1}A^{-1} = (A(I-A^{-1}B))^{-1} = (A-B)^{-1}$, which is the thesis. □

Lemma 5: Let us be given a matrix G with $\gamma = \|G\|$, and an orthogonal matrix O. If $0 < \alpha < (1/(1+\gamma))$, then the matrix $\alpha G + (1-\alpha)O$ is invertible.

Proof: I will prove that the kernel of $\alpha G + (1-\alpha)O$ is null. Assume there exists a vector $v \neq 0$ such that $[\alpha G + (1-\alpha)O]v = 0$. Then, it follows that $(1-\alpha)Ov = -\alpha Gv$. Now, by hypothesis $\alpha\gamma < (1-\alpha)$, hence, we have that $\|-\alpha Gv\| \leq \alpha\gamma\|v\| < (1-\alpha)\|v\| = \|(1-\alpha)Ov\|$. This contradicts the assumption $(1-\alpha)Ov = -\alpha Gv$. □

Lemma 6: Consider model (19) with activation function ϕ such that its derivative is bounded in $[0, r]$, and parameterized via $\alpha = \rho(L-1)^{-1}$. Denote $\sigma = \|W_h\|$. If $\rho < ((L-1)/(1+r\sigma))$, then for each $l = 0, \ldots, L-2$, the matrix $(\partial x_{l+1}/\partial x_l) = \alpha\operatorname{diag}(\phi'(y_l))W_h + (1-\alpha)O$ is invertible and its inverse admits the following upper bound $\|((\partial x_{l+1}/\partial x_l))^{-1}\| \leq (1/(1-\alpha(1+r\sigma)))$.

Proof: I will use the expression of Lemma 4, $(A-B)^{-1} = \sum_{k=0}^{\infty}(A^{-1}B)^kA^{-1}$, with $B = -\alpha\operatorname{diag}(\phi'(y_l))W_h$, and $A = (1-\alpha)O$, for which I know that $A^{-1} = (1-\alpha)^{-1}O^{-1}$. Therefore, by the triangle inequality, I know that $\|(\partial x_{l+1}/\partial x_l)^{-1}\| = \|\sum_{k=0}^{\infty}(A^{-1}B)^kA^{-1}\| \leq \sum_{k=0}^{\infty}\|(A^{-1}B)^kA^{-1}\|$. Then, sub-multiplicativity holds $\sum_{k=0}^{\infty}\|(A^{-1}B)^kA^{-1}\| \leq \sum_{k=0}^{\infty}\|(A^{-1}B)^k\|\|A^{-1}\|$. Again, sub-multiplicativity implies $\|(A^{-1}B)^k\| \leq \|(A^{-1}B)\|^k \leq \|A^{-1}\|^k\|B\|^k = (1/(1-\alpha)^k)\|B\|^k \leq (1/(1-\alpha)^k)(\alpha r\sigma)^k$. All together, we have the bound $\|(\partial x_{l+1}/\partial x_l)^{-1}\| \leq (1-\alpha)^{-1}\sum_{k=0}^{\infty}(\alpha r\sigma/(1-\alpha))^k$. The right term of the above inequality converges to $(1/(1-\alpha(1+r\sigma)))$, as long as $(\alpha r\sigma/(1-\alpha)) < 1$, that is, when $\alpha(1+r\sigma) < 1$. Assuming $\alpha = \rho(L-1)^{-1}$, the condition for the convergence is equivalent to $\rho < ((L-1)/(1+r\sigma))$. □

Theorem 4: Let us consider an roaRNN (19) with an activation function ϕ such that its derivative assumes values in $[0, r]$, and let us denote $\sigma = \|W_h\|$. If we parametrize $\alpha = \rho(L-1)^{-1}$, with a positive real-valued ρ such that $\rho < ((L-1)/(1+r\sigma))$, then matrices $(\partial x_L/\partial x_{s+1})$ are invertible for each $s = 0, \ldots, L-2$, and it holds the following L-independent lower bound for the minimum singular value of the IOJ $\mathrm{e}^{-\rho(1+r\sigma)} \leq \sigma_{\min}(\partial x_L/\partial x_1)$.

Proof: First of all, it is known that for any invertible matrix J, it holds that $\sigma_{\min}(J) = (1/\|J^{-1}\|)$. In our

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

case, we deal with $J = (\partial x_L/\partial x_1)$. I will find an upper bound for $\|J^{-1}\|$, which, in turn, will lead to a lower bound for $\sigma_{\min}(J)$. By definition, J is the product $[\alpha \operatorname{diag}(\phi'(y_{L-1}))W_h + (1-\alpha)O] \cdots [\alpha \operatorname{diag}(\phi'(y_1))W_h + (1-\alpha)O]$. Note that by Lemma 5, we have that J is a product of invertible matrices, and thus an invertible matrix itself. Now, $\|J^{-1}\| = \|(\prod_{l=L-1}^{1}(\partial x_{l+1}/\partial x_l))^{-1}\| = \|\prod_{l=1}^{L-1}(\partial x_{l+1}/\partial x_l)^{-1}\| \leq \prod_{l=1}^{L-1}\|(\partial x_{l+1}/\partial x_l)^{-1}\| \leq (1/[1-\alpha(1+r\sigma)]^{L-1}) \leq \mathrm{e}^{\rho(1+r\sigma)}$, where the first inequality is by sub-multiplicativity, the second inequality is Lemma 6, and the last is a known inequality of the exponential. Finally, we get the thesis $\sigma_{\min}(J) = (1/\|J^{-1}\|) \geq \exp(-\rho(1+r\sigma))$. □

APPENDIX C
REDUCING MEMORY AND TIME CONSUMPTION

Model (12) has twice more real numbers to store than a standard MLP. Two strategies to allay this issue are proposed.

1) Reduce memory and computation time consumption replacing random semiorthogonal matrices with random *semipermutation matrices* (see Definition 3). From preliminary experiments, randomly generated semipermutation matrices seem to work fine.
2) Recycle the same semiorthogonal matrix whenever dimensions permit it. For instance, the same orthogonal matrix might be exploited in an FNN having all layers of the same dimension from input to output.

Definition 3: A matrix $P \in \mathbb{R}^{N\times M}$ is a *semipermutation* if $P = [e_{\sigma(1)}, e_{\sigma(2)}, \ldots, e_{\sigma(M)}]$ (or $P = [e_{\sigma(1)}, e_{\sigma(2)}, \ldots, e_{\sigma(N)}]^{\mathrm{T}}$), where σ represents a permutation of N elements, if $N > M$, (or M elements, if $N \leq M$), and $e_{\sigma(j)}$ is the $\sigma(j)$th column of the identity matrix of dimension N (or M).

ACKNOWLEDGMENT

The author would like to personally thank Lorenzo Livi, Peter Ashwin, and Claudio Gallicchio, for their insightful comments and intellectual guidance.

REFERENCES

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986.
[3] D. Rumelhart, G. Hinton, and R. J. Williams, "Learning internal representations by error propagation, parallel distributed processing, explorations in the microstructure of cognition, ed. De Rumelhart and J. Mcclelland. vol. 1. 1986," *Biometrika*, vol. 71, nos. 599–607, p. 6, 1986.
[4] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Netw.*, vol. 5, no. 2, pp. 157–166, Mar. 1994.
[5] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 13th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.
[6] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.
[7] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 315–323.
[8] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," 2013, *arXiv:1312.6120*.
[9] N. Bertschinger and T. Natschläger, "Real-time computation at the edge of chaos in recurrent neural networks," *Neural Comput.*, vol. 16, no. 7, pp. 1413–1436, Jul. 2004.
[10] J. Pennington, S. S. Schoenholz, and S. Ganguli, "Resurrecting the sigmoid in deep learning through dynamical isometry: Theory and practice," 2017, *arXiv:1711.04735*.
[11] D. Mishkin and J. Matas, "All you need is a good init," 2015, *arXiv:1511.06422*.
[12] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2013, pp. 1310–1318.
[13] M. Liu, L. Chen, X. Du, L. Jin, and M. Shang, "Activated gradients for deep neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 2156–2168, Apr. 2023.
[14] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[15] R. Kumar Srivastava, K. Greff, and J. Schmidhuber, "Highway networks," 2015, *arXiv:1505.00387*.
[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[17] S. De and S. Smith, "Batch normalization biases residual blocks towards the identity function in deep networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, Jan. 2020, pp. 19964–19975.
[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[19] K. Cho, B. van Merrienboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," 2014, *arXiv:1409.1259*.
[20] M. Chen, J. Pennington, and S. S. Schoenholz, "Dynamical isometry and a mean field theory of RNNs: Gating enables signal propagation in recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 872–881.
[21] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1120–1128.
[22] S. Wisdom, T. Powers, J. R. Hershey, J. Le Roux, and L. Atlas, "Full-capacity unitary recurrent neural networks," 2016, *arXiv:1611.00035*.
[23] M. Lezcano-Casado and D. Martínez-Rubio, "Cheap orthogonal constraints in neural networks: A simple parametrization of the orthogonal and unitary group," in *Proc. Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 3794–3803.
[24] K. Helfrich, D. Willmott, and Q. Ye, "Orthogonal recurrent neural networks with scaled Cayley transform," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1969–1978.
[25] G. Kerg et al., "Non-normal recurrent neural network (nnRNN): Learning long time dependencies while improving expressivity with transient dynamics," 2019, *arXiv:1905.12080*.
[26] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," 2019, *arXiv:1901.02358*.
[27] S. Li, W. Li, C. Cook, C. Zhu, and Y. Gao, "Independently recurrent neural network (IndRNN): Building a longer and deeper RNN," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5457–5466.
[28] M. Lukoševičius and H. Jaeger, "Reservoir computing approaches to recurrent neural network training," *Comput. Sci. Rev.*, vol. 3, no. 3, pp. 127–149, Aug. 2009.
[29] B. Chang, M. Chen, E. Haber, and E. H. Chi, "AntisymmetricRNN: A dynamical system view on recurrent neural networks," 2019, *arXiv:1902.09689*.
[30] N. B. Erichson, O. Azencot, A. Queiruga, L. Hodgkinson, and M. W. Mahoney, "Lipschitz recurrent neural networks," 2020, *arXiv:2006.12070*.
[31] T. Konstantin Rusch and S. Mishra, "Coupled oscillatory recurrent neural network (coRNN): An accurate and (gradient) stable architecture for learning long time dependencies," 2020, *arXiv:2010.00951*.
[32] A. R. Voelker, I. Kajić, and C. Eliasmith, "Legendre memory units: Continuous-time representation in recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, Sep. 2019, pp. 15544–15553.
[33] S. Chandar, C. Sankar, E. Vorontsov, S. E. Kahou, and Y. Bengio, "Towards non-saturating recurrent units for modelling long-term dependencies," in *Proc. AAAI Conf. Artif. Intell.*, Jan. 2019, vol. 33, no. 1, pp. 3280–3287.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.

[34] Z. Quan, W. Zeng, X. Li, Y. Liu, Y. Yu, and W. Yang, "Recurrent neural networks with external addressable long-term and working memory for learning long-term dependences," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 3, pp. 813–826, Mar. 2020.

[35] S. Chang et al., "Dilated recurrent neural networks," 2017, *arXiv:1710.02224*.

[36] Y. Wang and F. Tian, "Recurrent residual learning for sequence classification," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2016, pp. 938–943.

[37] B. Yue, J. Fu, and J. Liang, "Residual recurrent neural networks for learning sequential representations," *Information*, vol. 9, no. 3, p. 56, Mar. 2018.

[38] A. Ravikumar and H. Sriraman, "Mitigating vanishing gradient in SGD optimization in neural networks," in *Proc. Int. Conf. Inf., Commun. Comput. Technol.* Cham, Switzerland: Springer, Jan. 2023, pp. 1–11.

[39] L. Borawar and R. Kaur, "ResNet: Solving vanishing gradient in deep networks," in *Proc. Int. Conf. Recent Trends Comput. (ICRTC)*. Cham, Switzerland: Springer, Jan. 2023, pp. 235–247.

[40] C. L. Galimberti, L. Furieri, L. Xu, and G. Ferrari-Trecate, "Hamiltonian deep neural networks guaranteeing nonvanishing gradients by design," *IEEE Trans. Autom. Control*, vol. 68, no. 5, pp. 3155–3162, May 2023.

[41] A. Ravikumar and H. Sriraman, "A deep understanding of long short-term memory for solving vanishing error problem: LSTM-VGP," in *Machine Learning Algorithms Using Scikit and TensorFlow Environments*. Hershey, PA, USA: IGI Global, 2024, pp. 74–90.

[42] N. Zucchet and A. Orvieto, "Recurrent neural networks: Vanishing and exploding gradients are not the end of the story," 2024, *arXiv:2405.21064*.

[43] Y. Guo et al., "Take a shortcut back: Mitigating the gradient vanishing for training spiking neural networks," 2024, *arXiv:2401.04486*.

[44] F. Mezzadri, "How to generate random matrices from the classical compact groups," 2006, *arXiv:math-ph/0609050*.

[45] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., London, U.K.: Pearson, 2010.

[46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[47] A. Ceni, P. Ashwin, L. Livi, and C. Postlethwaite, "The echo index and multistability in input-driven recurrent neural networks," *Phys. D, Nonlinear Phenomena*, vol. 412, Nov. 2020, Art. no. 132609.

[48] H. Jaeger, M. Lukoševičius, D. Popovici, and U. Siewert, "Optimization and applications of echo state networks with leaky-integrator neurons," *Neural Netw.*, vol. 20, no. 3, pp. 335–352, Apr. 2007.

[49] A. Ceni and C. Gallicchio, "Orthogonality in additive reservoir computing," in *Proc. Eur. Symp. Artif. Neural Netw., Comput. Intell. Mach. Learn.*, 2022, pp. 1–6.

[50] A. Ceni and C. Gallicchio, "Edge of stability echo state network," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, May 29, 2024, doi: 10.1109/TNNLS.2024.3400045.

[51] A. Ceni and C. Gallicchio, "Residual echo state networks: Residual recurrent neural networks with stable dynamics and fast learning," *Neurocomputing*, vol. 597, Sep. 2024, Art. no. 127966.

[52] S. Li, W. Li, C. Cook, and Y. Gao, "Deep independently recurrent neural network (IndRNN)," 2019, *arXiv:1910.06251*.

[53] Q. V. Le, N. Jaitly, and G. E. Hinton, "A simple way to initialize recurrent networks of rectified linear units," 2015, *arXiv:1504.00941*.

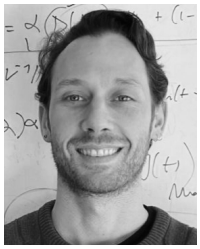
[54] M. Rotman and L. Wolf, "Shuffling recurrent neural networks," 2020, *arXiv:2007.07324*.

[55] T. Cooijmans, N. Ballas, C. Laurent, Ç. Gülçehre, and A. Courville, "Recurrent batch normalization," 2016, *arXiv:1603.09025*.

[56] A. Kag and V. Saligrama, "Time adaptive recurrent neural network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 15144–15153.

[57] P. Sun, J. Wu, M. Zhang, P. Devos, and D. Botteldooren, "Delayed memory unit: Modeling temporal dependency through delay gate," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Nov. 19, 2024, doi: 10.1109/TNNLS.2024.3490833.

[58] Ł. Neumann, Ł. Lepak, and P. Wawrzyński, "Least redundant gated recurrent neural network," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jun. 2023, pp. 1–10.



Andrea Ceni received the Ph.D. degree from the Department of Computer Science, University of Exeter, Exeter, U.K., in 2021.

He has been a Post-Doctoral Research Associate with CEMPS, University of Exeter. He is currently a Research Fellow with the Department of Computer Science, University of Pisa, Pisa, Italy.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:13:19 UTC from IEEE Xplore. Restrictions apply.